



به نام خدا
دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر



مهسا ابرغانی اقدم - ۸۱۰۱۰۳۰۵۳ - تمرین دوم - درس سامانه‌های یادگیری ماشین توزیع شده

سوال (۱)

الف) برای بخش الف در محل مشخص شده یک کد برای تصاویر از RGB به مقیاس خاکستری (Grayscale) نوشته شده است. هر تصویر رنگی از سه لایه تشکیل شده است. برای تبدیل آن به مقیاس خاکستری، مقدار هر پیکسل را در هر سه لایه بدست آورده و به فرمول مشخص شده می‌دهیم. خروجی این رابطه بعنوان مقدار آن پیکسل در مقیاس خاکستری خواهد بود.

```
# Define the RGB to Grayscale function
def rgb2grey_py(x):
#####
#WRITE YOUR PYHTON CODE HERER
r, g, b = image[0, :, :], image[1, :, :], image[2, :, :]
grey_image = 0.2989 * r + 0.5870 * g + 0.1140 * b
return grey_image
```

ب) در این بخش باید یک Kernel CUDA بنویسیم تا تصویر را به سیاه و سفید تبدیل کند. برای این کار ابتدا عملیات انجام شده بر روی کارت گرافیک را با زبان پایتون شبیه‌سازی می‌کنیم.

تابع اصلی blk_kernel:

این تابع نقش یک حلقه دو سطحی را برای شبیه‌سازی ساختار بلوک‌ها و threads در CUDA بازی می‌کند. به هر بلوک و ترد اجازه می‌دهد تابع f را برای مقادیر مشخصی از شاخص‌ها اجرا کند. پارامترها:

Blocks: تعداد بلوک‌ها.

Threads: تعداد تردها در هر بلوک.

f: تابعی که باید اجرا شود.

*args: آرگومان‌های اضافه که به تابع f ارسال می‌شوند.

تابع rgb2gray_bk:

این تابع برای پردازش یک بلوک از داده‌ها طراحی شده است.

با استفاده از شاخص‌های بلوک (blockidx) و ترد (threadidx)

موقعیت داده‌ی مورد نظر محاسبه می‌شود.

فرمول استاندارد تبدیل به Grayscale برای این داده اعمال می‌شود.

تابع rgb2gray_pybk:

این تابع وظیفه مدیریت داده‌های تصویری و فراخوانی عملیات بلوکی را بر عهده دارد.

مراحل:

اندازه‌گیری ابعاد تصویر (رنگ‌ها، ارتفاع و عرض).

محاسبه تعداد بلوک‌ها و تردها برای پردازش تصویر.
تبدیل تصویر به آرایه‌ای یک‌بعدی برای پردازش.
تخصیص فضای خروجی برای ذخیره تصویر Grayscale
اجرای blk_kernel برای اعمال تبدیل به مقیاس خاکستری.
بازگرداندن نتیجه در قالب اصلی تصویر (دو‌بعدی).

```
import math
def blk_kernel(f, blocks, threads, *args):
    for i in range(blocks):
        for j in range(threads): f(i, j, threads, *args)

def rgb2gray_bk(blockidx, threadidx, blockidm, x, out, n):
    i = blockidx * blockidm + threadidx
    if i < n: out[i] = 0.2989 * x[i] + 0.5870 * x[i+n] + 0.1140 * x[2*n+i]

def rgb2gray_pybk(x):
    c, h, w = x.shape
    n = h*w
    x = x.flatten()
    res = torch.empty(n, dtype=x.dtype, device=x.device)
    threads = 256
    blocks = int(math.ceil(h*w / threads))
    blk_kernel(rgb2gray_bk, blocks, threads, x, res, n)
    return res.view(h,w)

img_g = rgb2gray_pybk(img)
show_image(img_g, cmap='gray')]
```

با استفاده از این شبیه‌سازی انجام شده و دستیار هوش مصنوعی (chatGPT)، این کد را به C++ تبدیل می‌کنیم تا در قسمت مشخص شده برای کد cuda برای کامپایل قرار گیرد.

```

__global__ void rgb2gray_kernel(const unsigned char* x, unsigned char* out, int n, int h, int w) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        int pixel_base = idx;
        out[idx] = static_cast<unsigned char>(
            0.2989f * x[pixel_base] +
            0.5870f * x[pixel_base + n] +
            0.1140f * x[pixel_base + 2 * n]
        );
    }
}

torch::Tensor rgb_to_grayscale(torch::Tensor input) {
    CHECK_INPUT(input);
    int c = input.size(0);
    int h = input.size(1);
    int w = input.size(2);
    int n = h * w; // Total number of pixels

    auto input_flattened = input.flatten(1).contiguous();
    auto output = torch::empty({h, w}, torch::dtype(input.dtype()).device(input.device()));

    const int threads = 256;
    const int blocks = (n + threads - 1) / threads;

    rgb2gray_kernel<<<blocks, threads>>>(
        input_flattened.data_ptr<unsigned char>(),
        output.data_ptr<unsigned char>(),
        n, h, w
    );

    cudaDeviceSynchronize();

    return output;
}

```

کد `cuda` نوشته شده همان مراحل شبیه‌سازی شده در کد پایتون را انجام می‌دهد.

(ج) برای مقایسه روش ساده و روش استفاده از کد `cuda` از نظر زمان پردازشی، هردو روش مورد بررسی قرار گرفت که نتایج در ادامه تشریح می‌شود.

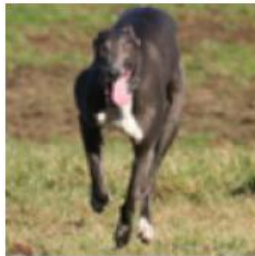
در استفاده از روش ساده زمان بدست آمده ۰.۲۴۳۲ ثانیه است و ۴ نمونه از تصاویر خواسته شده چاپ شده است.

Using device: cuda

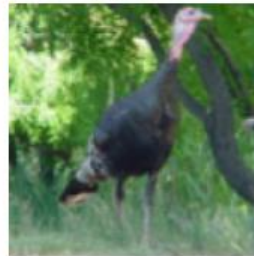
Label: 1



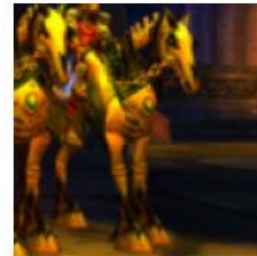
Label: 5



Label: 1



Label: 6

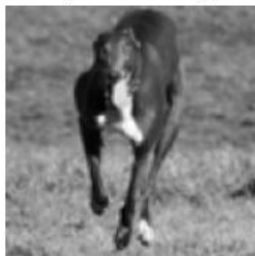


Time taken for Python-based grayscale conversion: 0.2432 seconds

Grayscale Image 1



Grayscale Image 2



Grayscale Image 3



Grayscale Image 4



در استفاده از روش استفاده از کد cuda زمان بدست آمده ۰.۰۸۱۲ ثانیه است و ۴ نمونه از تصاویر خواسته شده چاپ شده است.

Using device: cuda

Files already downloaded and verified

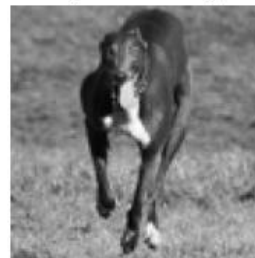
Time taken for Python-based grayscale conversion: 0.0812 seconds

3
3
3
3

Grayscale Image 1



Grayscale Image 2



Grayscale Image 3



Grayscale Image 4



روش شبیه‌سازی CUDA با بهره‌گیری از تقسیم وظایف و پردازش موازی، زمان پردازش را به‌طور چشمگیری کاهش داده است. برای حجم‌های بزرگ‌تر داده (مثلاً مجموعه تصاویر با حجم بالا)، تفاوت زمانی بین این دو روش حتی بیشتر خواهد بود. در مواردی که منابع GPU در دسترس باشند، استفاده از پیاده‌سازی موازی مانند CUDA یا کتابخانه‌های تخصصی مانند PyTorch، انتخاب بهتری برای پردازش تصاویر است.

هر دو روش توانستند تصاویر تبدیل‌شده به مقیاس خاکستری را به درستی تولید کنند و نمایش دهند. تعداد و کیفیت تصاویر نمایش‌داده‌شده مشابه بوده و صحت عملکرد هر دو روش تأیید شده است.

سوال ۲)

الف) هدف این تمرین آموزش یک مدل شبکه عصبی کانولوشن (CNN) است که بر روی دیتاست STL-10 که مجموعه‌ای از تصاویر رنگی 96×96 پیکسلی است، آموزش داده می‌شود. مدل به کمک PyTorch پیاده‌سازی شده است و از GPU برای تسریع فرایند آموزش استفاده می‌شود.

۱- وارد کردن کتابخانه‌ها: ابتدا کتابخانه‌های ضروری PyTorch و Torchvision برای مدل‌سازی و پردازش داده‌ها وارد می‌شوند. همچنین، کتابخانه time برای اندازه‌گیری زمان آموزش مورد استفاده قرار می‌گیرد.

۲- تنظیم دستگاه: کد تشخیص می‌دهد که آیا GPU در دسترس است یا خیر و از آن برای آموزش مدل استفاده می‌کند. در غیر این صورت، آموزش روی CPU انجام می‌شود.

```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as dsets
import time

device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(torch.cuda.is_available())
print(torch.cuda.current_device())
print(torch.cuda.device(0))
device = "cuda:0"
```

۳- بارگذاری داده‌ها: دیتاست STL-10 از مسیر مشخص شده بارگذاری می‌شود. داده‌ها به صورت اتوماتیک دانلود شده و سپس به قالب مناسب تبدیل می‌شوند.

```
train_set = dsets.STL10(data_path, split="train", transform=transforms.ToTensor(), download=True)
test_set = dsets.STL10(data_path, split="test", transform=transforms.ToTensor(), download=True)

train_loader = torch.utils.data.DataLoader(dataset=train_set, batch_size=batch_size, shuffle=True)
test_loader = torch.utils.data.DataLoader(dataset=test_set, batch_size=batch_size)
```

۴- تعریف مدل: مدل CNN به نام STL10CNN تعریف می‌شود که شامل ۳ لایه کانولوشن با MaxPooling و BatchNorm است. در نهایت، لایه‌های Fully Connected برای پیش‌بینی کلاس‌ها به کار می‌روند.

لایه اول: ۳۲ کانال کانولوشنی با سایز 3×3 .

لایه دوم: ۶۴ کانال کانولوشنی با سایز 3×3 .

لایه سوم: ۱۲۸ کانال کانولوشنی با سایز 3×3 .

پس از این لایه‌ها، داده‌ها فلت شده و به لایه‌های Fully Connected ارسال می‌شوند.

```

class STL10CNN(nn.Module):
    def __init__(self):
        super(STL10CNN, self).__init__()

        self.layer1 = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2))

        self.layer2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2))

        self.layer3 = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2))

        self.fc1 = nn.Linear(12 * 12 * 128, 256)
        self.drop = nn.Dropout(0.25)
        self.fc2 = nn.Linear(256, 10)

        nn.init.xavier_uniform_(self.fc1.weight)
        nn.init.xavier_uniform_(self.fc2.weight)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = self.layer3(out)
        out = out.view(out.size(0), -1)
        out = self.fc1(out)
        out = self.drop(out)
        out = self.fc2(out)
        return out

```

۵-تعریف تابع هزینه و بهینه‌ساز: از تابع CrossEntropyLoss برای محاسبه خطا و از بهینه‌ساز Adam برای بروزرسانی وزن‌ها استفاده می‌شود.

```

criterion = nn.CrossEntropyLoss().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

```

۶- آموزش مدل: آموزش مدل در چندین دوره انجام می‌شود. در هر دوره، مدل برای هر دسته از داده‌ها پیش‌بینی انجام داده و خطا (loss) را محاسبه می‌کند. پس از آن، از `loss.backward()` برای محاسبه گرادیان‌ها و از `optimizer.step()` برای بروزرسانی وزن‌ها استفاده می‌شود.

- هر ۵۰ تکرار، دقت مدل روی داده‌های تست محاسبه شده و در لیست‌ها ذخیره می‌شود.

- هر ۵۰۰ تکرار، دقت و ضرر مدل در کنسول چاپ می‌شود.

```
for epoch in range(training_epochs):
    avg_cost = 0

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

    count += 1
    if count % 50 == 0:
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            predictions = torch.max(outputs, 1)[1]
            correct += (predictions == labels).sum().item()
            total += labels.size(0)

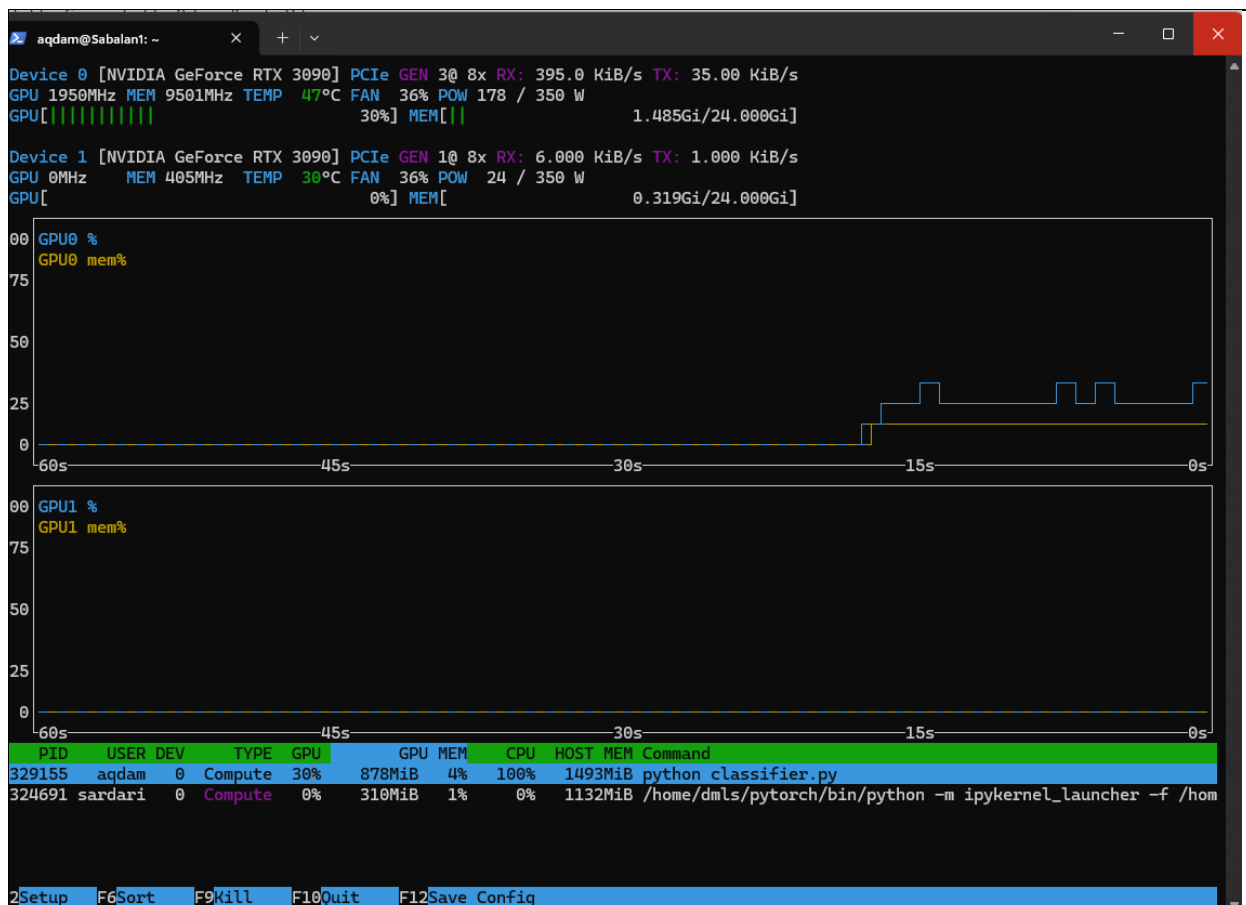
        accuracy = correct * 100 / total
        loss_list.append(loss.item())
        iteration_list.append(count)
        accuracy_list.append(accuracy)
    if count % 500 == 0:
        print(f"Iteration: {count}, Loss: {loss.item():.4f}, Accuracy: {accuracy:.2f}%")
```

۷- محاسبه زمان آموزش و مصرف حافظه: در نهایت، زمان آموزش و میزان حافظه مصرفی GPU ثبت و چاپ می‌شود. در پایان آموزش، دقت کلی مدل بر اساس مقادیر جمع‌آوری‌شده از طی مراحل تست محاسبه می‌شود.

```
cuda_mem = torch.cuda.max_memory_allocated(device)
print(f"Training time: {end_time - start_time:.2f}s")
print(f"Cuda Memory Usage: {cuda_mem / (1024 ** 2):.2f} MB")

final_accuracy = sum(accuracy_list) / len(accuracy_list)
print(f"Final Accuracy: {final_accuracy:.2f}%")
```

نتایج: با اندازه دسته ۳۲ و پردازش بر روی یک گرافیک نتایج زیر بدست آمده است.



الف) نمایش وضعیت سیستم و دستور در حال اجرا بر روی یک گرافیک

```
(pytorch) aqdam@Sabalan1:~$ python classifier.py
True
0
<torch.cuda.device object at 0x7f21cabadc0>
Files already downloaded and verified
Files already downloaded and verified
Iteration: 500, Loss: 2.1876, Accuracy: 38.34%
Iteration: 1000, Loss: 0.8315, Accuracy: 49.52%
Iteration: 1500, Loss: 0.6594, Accuracy: 56.90%
Iteration: 2000, Loss: 0.3771, Accuracy: 58.41%
Training time: 127.63s
Cuda Memory Usage: 446.53 MB
Final Accuracy: 50.65%
(pytorch) aqdam@Sabalan1:~$ |
```

ب) نتایج حاصله برای مدل معرفی شده در بخش الف سوال ۲

سوال ۲) ب) این کد به منظور آموزش یک مدل شبکه عصبی کانولوشن (CNN) با استفاده از PyTorch و پردازش موازی بر روی دیتاست STL-10 طراحی شده است. از تکنیک‌های پردازش توزیع‌شده (Distributed Data Parallel) برای تسریع آموزش بر روی چندین GPU استفاده می‌شود.

۱- تنظیم پردازش‌های توزیع‌شده: این بخش برای راه‌اندازی پردازش‌های توزیع‌شده و تعیین پیکربندی شبکه مانند آدرس و پورت سرور و تنظیم پارامترهای مربوط به پردازش توزیع‌شده استفاده می‌شود.

۲- یافتن پورت آزاد برای ارتباط بین پردازش‌ها: برای اطمینان از عدم تداخل پورت‌ها، یک پورت آزاد پیدا می‌شود که پردازش‌های مختلف بتوانند از آن استفاده کنند.

```
def setup(rank, world_size, master_port, backend, timeout):
    os.environ['MASTER_ADDR'] = 'localhost'
    os.environ['MASTER_PORT'] = master_port
    dist.init_process_group(backend, rank=rank, world_size=world_size, timeout=timeout)

def find_free_port():
    import socket
    from contextlib import closing

    with closing(socket.socket(socket.AF_INET, socket.SOCK_STREAM)) as s:
        s.bind(('', 0))
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        return str(s.getsockname()[1])
```

۳- بارگذاری داده‌ها و توزیع آن‌ها: در این قسمت، داده‌ها با استفاده از DistributedSampler توزیع شده و به هر پردازش اختصاص داده می‌شود. این کار باعث می‌شود که داده‌ها بین GPU های مختلف تقسیم شوند و هر پردازش فقط قسمتی از داده‌ها را پردازش کند.

```
def load_data(rank, world_size):
    train_set = datasets.STL10(data_path, split="train", transform=transforms.Compose([transforms.ToTensor()]), download=True)
    train_sampler = torch.utils.data.distributed.DistributedSampler(
        train_set, num_replicas=world_size, rank=rank
    )
    train_loader = torch.utils.data.DataLoader(
        dataset=train_set, sampler=train_sampler, batch_size=batch_size,
        persistent_workers=True, num_workers=1, pin_memory=True
    )

    test_set = datasets.STL10(data_path, split="test", transform=transforms.Compose([transforms.ToTensor()]), download=True)
    test_sampler = torch.utils.data.distributed.DistributedSampler(
        test_set, num_replicas=world_size, rank=rank
    )
    test_loader = torch.utils.data.DataLoader(
        dataset=test_set, sampler=test_sampler, batch_size=batch_size,
        persistent_workers=True, num_workers=1, pin_memory=True
    )

    return train_loader, test_loader
```

۴- آموزش مدل با استفاده از پردازش توزیع‌شده: مدل به کمک DistributedDataParallel آموزش داده می‌شود. این تکنیک باعث می‌شود که مدل در هر پردازش به طور مستقل اجرا شده و سپس وزن‌ها با یکدیگر همگام شوند.

۵-ارزیابی مدل و محاسبه دقت: هر ۵۰ تکرار، مدل بر روی داده‌های تست ارزیابی می‌شود و دقت آن محاسبه می‌شود. دقت‌ها و ضررها در لیست‌ها ذخیره شده و چاپ می‌شوند.

```
def train(rank, world_size, master_port, backend, timeout):
    setup(rank, world_size, master_port, backend, timeout)
    torch.cuda.set_device(rank)
    train_loader, test_loader = load_data(rank, world_size)
    model = STL10CNN().to(rank)
    ddp_model = DDP(model, device_ids=[rank])
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(ddp_model.parameters(), lr=learning_rate)

    count = 0
    loss_list = []
    iteration_list = []
    accuracy_list = []
    predictions_list = []
    labels_list = []

    start_time = time.time()
    for epoch in range(training_epochs):
        for images, labels in train_loader:
            images, labels = images.to(rank), labels.to(rank)

            optimizer.zero_grad()
            outputs = ddp_model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            count += 1
            if count % 50 == 0:
                total = 0
                correct = 0
                for images, labels in test_loader:
                    images, labels = images.to(rank), labels.to(rank)
                    labels_list.append(labels)
                    outputs = ddp_model(images)
                    predictions = torch.max(outputs, 1)[1].to(rank)
                    labels_list.append(labels)
                    correct += (predictions == labels).sum().item()
                    predictions_list.append(predictions)
                    correct += (predictions == labels).sum()
                    total += labels.size(0)

                accuracy = correct * 100 / total
                loss_list.append(loss.item())
                iteration_list.append(count)
                accuracy_list.append(accuracy)

            if count % 500 == 0:
                print(f"Iteration: {count}, Loss: {loss.item():.4f}, Accuracy: {accuracy:.2f}%")

    end_time = time.time()
    cuda_mem = torch.cuda.max_memory_allocated(rank)
    print(f"Rank: {rank}, Training time: {end_time - start_time:.2f}s")
    print(f"Rank: {rank}, Max Memory Allocated: {cuda_mem / (1024 ** 2):.2f} MB")

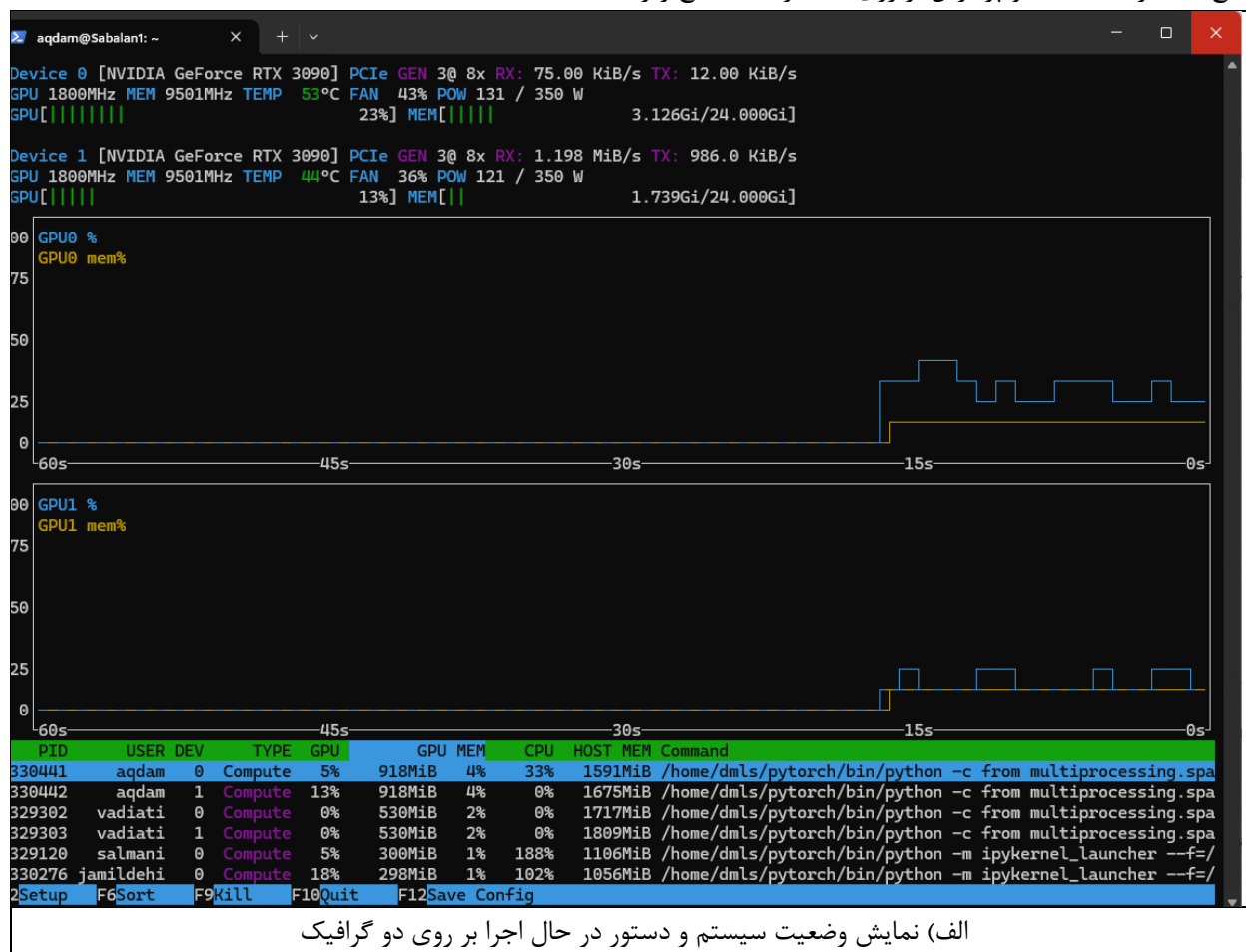
    final_accuracy = sum(accuracy_list) / len(accuracy_list)
    print(f"Rank: {rank}, Final Accuracy: {final_accuracy:.2f}%")
```

آغاز پردازش‌های موازی: با استفاده از `mp.spawn` آموزش به طور همزمان بر روی چندین پردازشگر و در هر پردازشگر به طور مستقل انجام می‌شود.

```
if __name__ == "__main__":
    world_size = torch.cuda.device_count()
    master_port = find_free_port()
    backends = ['gloo', 'nccl']
    timeout = datetime.timedelta(seconds=10)

    for backend in backends:
        print(f"Starting training with {backend}")
        start_time = time.time()
        mp.spawn(train, nprocs=world_size, args=(world_size, master_port, backend, timeout), join=True, start_method="spawn")
        end_time = time.time()
        print(f"Total time: {end_time - start_time:.2f}s")
```

نتایج: با اندازه دسته ۳۲ و پردازش بر روی یک گرافیک نتایج زیر بدست آمده است.



الف) نمایش وضعیت سیستم و دستور در حال اجرا بر روی دو گرافیک

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Iteration: 500, Loss: 1.5184, Accuracy: 82.15%
Iteration: 500, Loss: 1.9242, Accuracy: 85.75%
Iteration: 1000, Loss: 0.8143, Accuracy: 101.65%
Iteration: 1000, Loss: 0.5470, Accuracy: 103.55%
Rank: 0, Training time: 61.36s
Rank: 1, Training time: 61.36s
Rank: 0, Max Memory Allocated: 467.61 MB
Rank: 1, Max Memory Allocated: 467.61 MB
Rank: 0, Final Accuracy: 88.82%
Rank: 1, Final Accuracy: 88.11%
Total time: 72.38s
(ب) نتایج حاصله برای مدل معرفی شده در بخش ب سوال ۲

سوال ۲)

(ج) مقایسه نتایج:

ردیف	مدل	تعداد کارت گرافیک	اندازه دسته	دقت (درصد)	زمان اجرا (ثانیه)	حافظه درگیر شده (MB)
۱	الف	۱	۳۲	۵۰.۶۵	۱۲۷.۶۳	۴۴۶.۵۳
۲	ب	۲	۳۲	۸۸.۲۱ و ۸۸.۸۲	۷۲.۳۸	۴۷۶.۶۱ در هر گرافیک

دقت مدل:

افزایش تعداد کارت‌های گرافیک در آزمایش ب باعث افزایش قابل توجهی در دقت مدل نسبت به آزمایش الف شده است. این امر نشان می‌دهد که پردازش موازی روی چندین GPU می‌تواند باعث بهبود توانایی مدل در یادگیری و در نتیجه دقت نهایی آن شود. در آزمایش الف، تنها یک کارت گرافیک استفاده شده است که به دلیل محدودیت‌های منابع و پردازش، دقت کمتری حاصل شده است.

تحلیل زمان اجرا:

در آزمایش ب، با استفاده از دو کارت گرافیک، زمان آموزش کاهش قابل توجهی داشته است (تقریباً ۴۳.۲۵ ثانیه کمتر از آزمایش الف). این نشان می‌دهد که پردازش توزیع‌شده و استفاده از چندین GPU باعث کاهش زمان آموزش می‌شود.

تحلیل حافظه مصرفی:

حافظه مصرفی در آزمایش ب برای هر کارت گرافیک افزایش یافته است (تقریباً ۳۰ MB بیشتر از آزمایش الف). این امر ممکن است به دلیل توزیع مدل بین دو کارت گرافیک باشد که نیاز به حافظه بیشتری برای ذخیره‌سازی مدل و داده‌های توزیع شده دارد.

با این حال، این افزایش حافظه به نوعی معقول است زیرا تعداد پردازش‌ها و داده‌های مورد استفاده در هر کارت گرافیک به دلیل تقسیم بار پردازشی افزایش یافته است.

سوال (۳)

تحلیل دقت:

سایز کوچک‌تر باعث بهبود دقت مدل می‌شود. این امر نشان می‌دهد که با داشتن نمونه‌های بیشتر برای هر آپدیت پارامتر، مدل قادر است ویژگی‌های داده‌ها را دقیق‌تر یاد بگیرد.

در batch sizes بزرگتر، دقت مدل کاهش یافته است، به جز در سایز ۱۲۸ که دقت دوباره افزایش یافته است. این ممکن است به دلیل محدودیت‌های موجود در آموزش و یا اثرات نوسانات تصادفی در آپدیت‌های پارامترها باشد.

تحلیل زمان اجرا:

همانطور که پیش‌بینی می‌شد، با افزایش سایز batch، زمان اجرای آموزش کاهش یافته است. دلیل این امر این است که با استفاده از batch sizes بزرگتر، می‌توان چندین نمونه را به طور همزمان پردازش کرد و بنابراین نیاز به تعداد کمتری از آپدیت‌های پارامتر است.

تحلیل حافظه مصرفی:

با افزایش batch size، حافظه مصرفی نیز به طور خطی افزایش می‌یابد. دلیل این امر این است که با استفاده از batches بزرگتر، حافظه بیشتری برای ذخیره داده‌های ورودی و ویژگی‌ها در طی پردازش‌های موازی نیاز است.

ردیف	اندازه دسته	دقت (درصد)	زمان اجرا (ثانیه)	حافظه درگیرشده (MB)
۱	۱۶	۹۳	۱۲۳.۲۸	۲۹۹.۴۰
۲	۳۲	۸۸	۷۲.۳۸	۴۷۶.۶۱
۳	۶۴	۸۴	۴۷.۷۸	۸۱۹.۹۴
۴	۱۲۸	۸۹	۳۷.۹۹	۱۵۳۲.۳۹

اندازه دسته ۱۶:

```

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Iteration: 500, Loss: 1.1480, Accuracy: 88.70%
Iteration: 500, Loss: 1.3108, Accuracy: 87.30%
Iteration: 1000, Loss: 1.0057, Accuracy: 94.95%
Iteration: 1000, Loss: 1.2574, Accuracy: 97.05%
Iteration: 1500, Loss: 1.1200, Accuracy: 98.45%
Iteration: 1500, Loss: 1.3083, Accuracy: 97.40%
Iteration: 2000, Loss: 1.0818, Accuracy: 101.25%
Iteration: 2000, Loss: 0.2753, Accuracy: 103.15%
Rank: 1, Training time: 112.11s
Rank: 0, Training time: 112.11s
Rank: 1, Max Memory Allocated: 299.40 MB
Rank: 0, Max Memory Allocated: 299.40 MB
Rank: 0, Final Accuracy: 93.19%
Rank: 1, Final Accuracy: 93.57%
Total time: 123.28s

```

اندازه دسته ۳۲:

```

Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Iteration: 500, Loss: 1.5184, Accuracy: 82.15%
Iteration: 500, Loss: 1.9242, Accuracy: 85.75%
Iteration: 1000, Loss: 0.8143, Accuracy: 101.65%
Iteration: 1000, Loss: 0.5470, Accuracy: 103.55%
Rank: 0, Training time: 61.36s
Rank: 1, Training time: 61.36s
Rank: 0, Max Memory Allocated: 467.61 MB
Rank: 1, Max Memory Allocated: 467.61 MB
Rank: 0, Final Accuracy: 88.82%
Rank: 1, Final Accuracy: 88.11%
Total time: 72.38s

```

اندازه دسته ۶۴:

```
(pytorch) aqdam@Sabalan1:~$ python classifier_mp.py
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Iteration: 500, Loss: 0.9125, Accuracy: 96.55%
Iteration: 500, Loss: 0.7919, Accuracy: 97.15%
Rank: 0, Training time: 37.13s
Rank: 0, Max Memory Allocated: 819.94 MB
Rank: 1, Training time: 37.13s
Rank: 1, Max Memory Allocated: 819.94 MB
Rank: 0, Final Accuracy: 84.18%
Rank: 1, Final Accuracy: 83.17%
Total time: 47.78s
```

اندازه دسته ۱۲۸:

```
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Rank: 0, Training time: 26.90s
Rank: 0, Max Memory Allocated: 1532.39 MB
Rank: 1, Training time: 26.90s
Rank: 1, Max Memory Allocated: 1532.39 MB
Rank: 0, Final Accuracy: 89.21%
Rank: 1, Final Accuracy: 89.38%
Total time: 37.99s
```

سوال (۴)

تحلیل دقت:

با استفاده از gloo در اندازه دسته ۱۲۸، دقت مدل بهبود یافته است. این نشان می‌دهد که gloo ممکن است در پردازش داده‌های بزرگ‌تر بهتر عمل کند.

در nccl، کاهش دقت در اندازه دسته ۱۲۸ ممکن است ناشی از تفاوت‌های خاص در نحوه پردازش داده‌ها یا ارتباطات در شبکه باشد که باعث کاهش کارایی در مدل‌های با batch size بزرگ‌تر می‌شود.

تحلیل زمان اجرا:

برای اندازه دسته ۳۲، nccl زمان کمتری برای آموزش نیاز دارد، که ممکن است به دلیل کارایی بالاتر در شبکه‌های توزیع‌شده در این backend باشد.

در اندازه دسته ۱۲۸، زمان آموزش به طور کلی کوتاه‌تر است و تفاوت‌های زمان اجرا بین gloo و nccl ناچیز است. این ممکن است به دلیل کاهش تعداد آپدیت‌های پارامتر در batchs بزرگ‌تر باشد.

تحلیل حافظه مصرفی:

حافظه مصرفی GPU به طور مستقل از backend انتخاب شده افزایش می‌یابد و تنها با توجه به سایز batch تغییر می‌کند. این نشان می‌دهد که پردازش داده‌های بزرگ‌تر نیاز به حافظه بیشتری دارد، که در هر دو backend به یک میزان مصرف می‌شود.

ردیف	backend	اندازه دسته	دقت (درصد)	زمان اجرا (ثانیه)	حافظه درگیر شده (MB)
۱	gloo	۳۲	۸۸	۷۲.۳۸	۴۷۶.۶۱
۲	nccl	۳۲	۸۸	۶۴.۴۸	۴۷۶.۶۱
۳	gloo	۱۲۸	۸۹	۳۷.۹۹	۱۵۳۲.۳۹
۴	nccl	۱۲۸	۸۲	۳۷.۴۵	۱۵۳۲.۳۹

اندازه دسته ۳۲:

```
(pytorch) aqdam@Sabalan1:~$ python classifier_mp.py
Starting training with gloo
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Iteration: 500, Loss: 1.5184, Accuracy: 82.15%
Iteration: 500, Loss: 1.9242, Accuracy: 85.75%
Iteration: 1000, Loss: 0.8143, Accuracy: 101.65%
Iteration: 1000, Loss: 0.5470, Accuracy: 103.55%
Rank: 0, Training time: 61.36s
Rank: 1, Training time: 61.36s
Rank: 0, Max Memory Allocated: 467.61 MB
Rank: 1, Max Memory Allocated: 467.61 MB
Rank: 0, Final Accuracy: 88.82%
Rank: 1, Final Accuracy: 88.11%
Total time: 72.38s
Starting training with nccl
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Iteration: 500, Loss: 0.9084, Accuracy: 92.00%
Iteration: 500, Loss: 1.3657, Accuracy: 91.70%
Iteration: 1000, Loss: 0.7291, Accuracy: 99.05%
Iteration: 1000, Loss: 0.6091, Accuracy: 100.20%
Rank: 0, Training time: 53.24s
Rank: 0, Max Memory Allocated: 467.61 MB
Rank: 1, Training time: 53.24s
Rank: 1, Max Memory Allocated: 467.61 MB
Rank: 0, Final Accuracy: 88.93%
Rank: 1, Final Accuracy: 88.36%
[rank0]:[W1130 00:24:09.483639318 ProcessGroupNCCL.cpp:1250] Warning: WARNING: process group has NOT been destroyed before we destruct ProcessGroupNCCL. On normal program exit, the application should call destroy_process_group to ensure that any pending NCCL operations have finished in this process. In rare cases this process can exit before this point and block the progress of another member of the process group. This constraint has always been present, but this warning has only been added since PyTorch 2.4 (function operator())
Total time: 64.48s
(pytorch) aqdam@Sabalan1:~$ |
```

اندازه دسته ۱۲۸:

```
(pytorch) aqdam@Sabalani:~$ python classifier_mp.py
Starting training with gloo
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Rank: 0, Training time: 26.90s
Rank: 0, Max Memory Allocated: 1532.39 MB
Rank: 1, Training time: 26.90s
Rank: 1, Max Memory Allocated: 1532.39 MB
Rank: 0, Final Accuracy: 89.21%
Rank: 1, Final Accuracy: 89.38%
Total time: 37.99s
Starting training with nccl
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
Rank: 0, Training time: 26.22s
Rank: 0, Max Memory Allocated: 1532.39 MB
Rank: 0, Final Accuracy: 81.87%
Rank: 1, Training time: 26.25s
Rank: 1, Max Memory Allocated: 1532.39 MB
Rank: 1, Final Accuracy: 82.02%
[rank0]:[W1130 00:28:39.048903499 ProcessGroupNCCL.cpp:1250] Warning: WARNING: process group has NOT been destroyed before we destruct ProcessGroupNCCL. On normal program exit, the application should call destroy_process_group to ensure that any pending NCCL operations have finished in this process. In rare cases this process can exit before this point and block the progress of another member of the process group. This constraint has always been present, but this warning has only been added since PyTorch 2.4 (function operator())
Total time: 37.45s
```